BRIEFING TÉCNICO · DOCUMENTAÇÃO DE SISTEMA

MP Conciliador

Plataforma web de conciliação financeira de marketplaces — integra Mercado Pago, Mercado Livre e Tiny (ERP) para reconciliar pagamentos, taxas, frete e notas fiscais. Como o sistema é construído e como ele funciona, por dentro.

Produto: MP Conciliador · **Contexto:** Control Up · **Pasta:** /mp_conciliador
Natureza: Documento técnico de arquitetura e funcionalidades · **Versão do documento:** 1.0

Sumário

1. 1. Visão geral
2. 2. Arquitetura
3. 3. Autenticação, acesso e multi-empresa
4. 4. Integrações externas
5. 5. Motor de sincronização do Mercado Pago (o coração)
6. 6. Conciliação de frete e bônus (Mercado Livre)
7. 7. Busca de notas fiscais no Tiny
8. 8. Módulos e telas
9. 9. Modelo de dados (banco)
10. 10. Relatórios, exportação e notificações
11. 11. Dependências e requisitos
12. 12. Limitações e observações de segurança

1. Visão geral

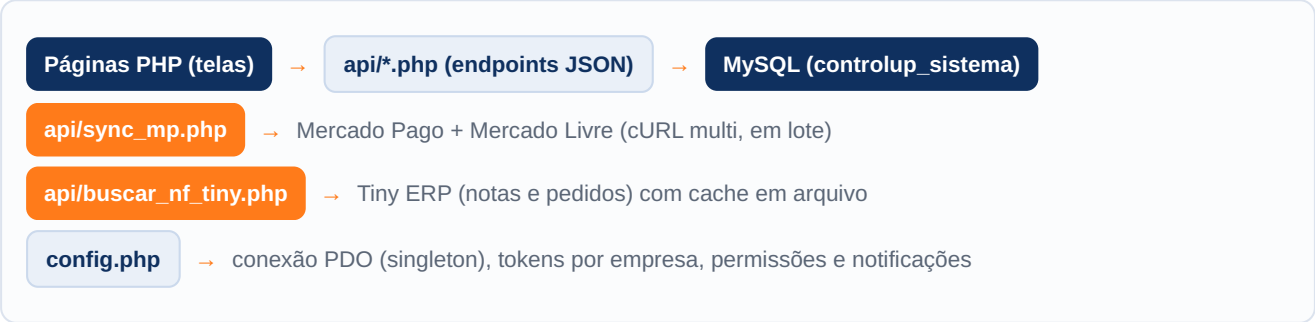
O **MP Conciliador** é um sistema web de **conciliação financeira de marketplaces**. Ele importa os pagamentos do **Mercado Pago**, enriquece cada venda com dados do **Mercado Livre** (pedido, frete, bônus) e cruza com as **notas fiscais do Tiny (ERP)**, consolidando tudo em painéis de conciliação, DRE de taxas, balancete diário, saldo e relatórios. É multiusuário e multiempresa, com login em duas etapas.

Tecnicamente é uma aplicação **PHP + MySQL** clássica (sem framework SPA): cada tela é uma página PHP server-side que consulta o banco e chama endpoints da pasta `api/`. A sincronização pesada com as APIs externas é feita por scripts que rodam sob demanda, usando **requisições paralelas** (cURL multi) para dar conta do volume dentro do tempo de execução.

> **Stack em uma linha**
>
> **Back/Front:** PHP 8 server-side (PDO/MySQL) + HTML/JS + Chart.js | **Banco:** MySQL (`controlup_sistema`) | **Integrações:** Mercado Pago, Mercado Livre, Tiny ERP, SendGrid (e-mail), Amazon SP-API (planejado) | **PDF:** Dompdf | **Auth:** sessão + 2FA por e-mail

2. Arquitetura

O sistema segue o modelo **página PHP + API interna + banco**, com integrações externas isoladas em endpoints próprios.



- **config.php** é o núcleo compartilhado: abre a conexão PDO (singleton), resolve a empresa ativa e seus tokens, define a matriz de permissões e as funções de notificação.
- **Páginas** (ex.: `conciliador.php`, `dre.php`, `balancete_diario.php`, `saldo.php`) verificam sessão/permissão no topo e renderizam o HTML com os dados do banco.
- **Pasta `api/`** concentra as ações: sincronizar, listar por data/liberação, detalhar pagamento, buscar NF, exportar PDF, autenticar, etc. — devolvem JSON para o front.
- **Roteamento:** `.htaccess` reescreve URLs amigáveis (ex.: `/login` → `login.php`) e desliga cache das páginas PHP.
- **Persistência auxiliar em arquivos** (`data/`): caches de NF e frete, logs de sync/busca e um `payments.json` legado mantido como backup.

3. Autenticação, acesso e multi-empresa

3.1 Login em duas etapas (2FA)

- **Etapa 1:** e-mail + senha; a senha é verificada com `password_verify` (hash bcrypt na tabela `usuarios`).
- **Etapa 2:** gera um **código de 6 dígitos**, grava em `codigos_verificacao` (expira em 5 min, máx. 5 tentativas) e envia por e-mail via **SendGrid** (PHPMailer/SMTP). Só então a sessão de login é criada.
- **Sessão:** guarda `usuario_id`, `nome`, `email` e `nivel`; há expiração por inatividade (ex.: 30 min em alguns módulos). Recuperação de senha por token (`reset_senha_tokens`).

3.2 Níveis de acesso e permissões

Três níveis — **admin**, **usuario** e **visualizador** — governados por uma matriz em `config.php` (`temPermissao` / `verificarAcesso`). Exemplos: DRE, fluxo de caixa, sugestões, configurações e gestão de empresas são restritos a admin; conciliação é liberada a todos; exportar e buscar NF ficam fora do visualizador.

3.3 Multi-empresa (multi-tenant)

Cada usuário pertence a uma ou mais empresas (`empresa_usuarios`), e a empresa ativa fica na sessão. Os **tokens de API (Mercado Pago, Mercado Livre, Tiny) são por empresa** — `getTokenMP/ML/Tiny()` leem da empresa ativa, com um *fallback* global. Trocar de empresa troca o conjunto de credenciais e os dados exibidos.

4. Integrações externas

Serviço	Endpoints usados	Para quê
Mercado Pago	`/v1/payments/search`, `/v1/payments/{id}`	Importar pagamentos do período (bruto, líquido, taxas, status, liberação) e detalhar um pagamento.
Mercado Livre	`/orders/{id}`, `/shipments/{id}`, `/shipments/{id}/costs`	Enriquecer a venda: pedido, envio e custos de frete / bônus do vendedor.
Tiny (ERP)	`api2/notas.fiscais.pesquisa`, `pedidos.pesquisa`, `pedido.obter`	Localizar a nota fiscal da venda (número, série, chave, valor, situação).
SendGrid	SMTP (PHPMailer)	Enviar o código 2FA e e-mails de recuperação de senha.
Amazon SP-API	SDK via Composer	Conciliação Amazon — previsto no menu como "Em breve".
Dompdf	biblioteca local	Gerar PDFs de relatórios e gráficos.

Autenticação nas APIs de marketplace é por **Bearer token** (cabeçalho `Authorization`); o Tiny usa token no corpo da requisição. As chamadas em volume usam `curl_multi` para rodar várias em paralelo.

5. Motor de sincronização do Mercado Pago (o coração)

O `api/sync_mp.php` é o processo central. Ele recebe um período (`begin_date` / `end_date`), busca os pagamentos no Mercado Pago e grava/atualiza no banco. Pontos técnicos relevantes:

- **Janela de tempo:** usa fuso `America/Sao_Paulo` e amplia a janela enviada à API em ±1 dia para não perder eventos de borda; registra um *diagnóstico* com os horários usados.
- **Paginação em paralelo:** percorre a busca em páginas (`offset` / `limit`) disparando vários `offsets` ao mesmo tempo (`curl_multi`), em lotes, até acabar ou atingir o teto de tempo.
- **Teto de tempo:** `set_time_limit(150)` e um corte seguro (~110 s) que encerra a rodada e sinaliza `atingiu_limite_tempo`, permitindo continuar depois sem travar.
- **Tratamento de frete:** transações `marketplace_shipment` **não viram registro próprio** — o custo de frete é extraído (`cost_components.ratio`) e aplicado ao pedido original, casando por `external_reference`.
- **Gravação (UPSERT):** `INSERT … ON DUPLICATE KEY UPDATE` na tabela `pagamentos`, atualizando status, taxas, líquido, data de liberação, frete, etc. As taxas detalhadas vão para `pagamento_taxas`.
- **Liberação do dinheiro:** calcula `money_release_date` e `dias_para_liberacao` para o fluxo de recebíveis.
- **Auto-migração de esquema:** `garantirColuna()` adiciona colunas novas na tabela `pagamentos` sob demanda (`ALTER TABLE … ADD COLUMN IF NOT EXISTS`) e grava um arquivo-marcador `.schema_col_*_ok` para não repetir.

> **Retorno do sync**
>
> Devolve um JSON com contadores da rodada: `inseridos`, `atualizados`, `total`, `bonus_frete_encontrados`, `frete_transactions_ignoradas`, `custos_frete_aplicados`, `tempo` e o `diagnostico` de horários.

6. Conciliação de frete e bônus (Mercado Livre)

Para refinar a margem real de cada venda, o sync cruza com o Mercado Livre em duas etapas, também em paralelo:

1. Para cada pagamento com envio, busca o **pedido** (`/orders/{order_id}`) e extrai o `shipping.id`.
2. Com o `shipping.id`, busca os **custos do envio** (`/shipments/{id}/costs`) e calcula o **bônus de frete**: quando o custo do vendedor é zero mas há desconto promovido ao comprador, o valor entra como bônus (`shipping_bonus`), marcado com `shipping_bonus_checked_at` para não reprocessar.

Há um teto de verificações de bônus por execução para controlar tempo e chamadas, e a lista de pagamentos que falharam é registrada para nova tentativa.

7. Busca de notas fiscais no Tiny

O `api/buscar_nf_tiny.php` localiza a NF de uma venda no Tiny usando **múltiplas estratégias em cascata**, porque nem toda venda aparece do mesmo jeito:

- **Estratégia 1 — NF direta:** `notas.fiscais.pesquisa` pelo `numeroEcommerce` (o id do pedido do marketplace).
- **Estratégia 2 — via pedido:** `pedidos.pesquisa` → `pedido.obter` para extrair a NF vinculada quando a busca direta falha.
- **Cache e log:** resultados ficam em `data/nf_cache.json` por 1 hora; buscas e NFs não encontradas são registradas em log para auditoria.

Retorna número, série, chave de acesso, data de emissão, valor e situação — o que fecha a conciliação entre o recebimento (MP), o pedido (ML) e o documento fiscal (Tiny).

8. Módulos e telas

Módulo	O que faz
Dashboard (`index.php`)	Visão inicial com indicadores e notificações não lidas.
Conciliação MP (`conciliador.php`)	Tela principal: sincroniza o período, lista pagamentos, taxas, frete, status e permite buscar a NF e detalhar cada pagamento.
Recebimentos do Dia MP	Calendário/gráfico de recebíveis por data de liberação.
DRE / Taxas MP (`dre.php`)	Demonstrativo de resultados a partir das taxas e valores do período (restrito).
Balancete Diário MP	Consolidação diária de valores (restrito).
Saldo MP (`saldo.php`)	Soma bruto × líquido dos pagamentos aprovados num período (consulta direta à API do MP).
Relatórios ML	Relatórios sobre dados do Mercado Livre.
Conciliação Amazon / Shopee	Previstas no menu como "Em breve".
Empresas	Troca de empresa ativa; gestão de empresas e vínculos (admin).
Administração	Gerenciar usuários, gerenciar empresas e criar notificações (admin).

Outros	Meu Perfil, Sugerir Melhoria, Notificações, Sair.

9. Modelo de dados (banco)

Banco MySQL `controlup_sistema`, acessado por PDO com *prepared statements*. Principais tabelas:

Tabela	Conteúdo
`pagamentos`	Núcleo financeiro: cada pagamento do MP (bruto, líquido, taxas somadas, frete, bônus, status, datas, liberação, empresa). Esquema evolui por auto-migração.
`pagamento_taxas`	Taxas detalhadas por pagamento (tipo, valor, pagador, categoria).
`empresas`	Empresas atendidas e seus tokens (MP/ML/Tiny) e dados cadastrais.
`usuarios`	Usuários, senha (bcrypt), nível e status.
`empresa_usuarios`	Vínculo usuário × empresa (com nível e ativo) — base do multi-tenant.
`notificacoes`	Notificações internas por usuário (título, mensagem, tipo, link, lida).
`codigos_verificacao`	Códigos 2FA (código, expiração, usado, tentativas).
`reset_senha_tokens`	Tokens de recuperação de senha.

Além do banco, a pasta `data/` guarda caches (`nf_cache.json` , `cache_frete` , `tiny_index`), logs (`sync_debug.log` , `nf_busca.log`) e o `payments.json` legado.

10. Relatórios, exportação e notificações

- **PDF:** `api/gerar_pdf.php` e `api/exportar_pdf_grafico.php` usam **Dompdf** para exportar relatórios e gráficos; dados de gráfico podem ser salvos antes da geração (`salvar_dados_pdf.php`).
- **Listagens:** `list_por_data.php` e `list_por_liberacao.php` alimentam as visões por data de venda e por data de liberação; `resumo_mes_liberacao.php` consolida o mês.
- **Notificações:** sistema interno completo (criar, listar, marcar lida / todas, ver) com contador de não lidas no topo; funções `criarNotificacao` e `notificarTodosUsuarios` em `config.php` .
- **Ferramentas de apoio:** `buscador_de_campo_api.php` (explorar campos crus das APIs MP/ML) e `debug.php` / `test.php` para diagnóstico.

11. Dependências e requisitos

- **PHP 8** com extensões `cURL` , `PDO/MySQL` e `GD` ; **MySQL** com o schema `controlup_sistema` .
- **Composer:** PHPMailer (e-mail) e o SDK Amazon SP-API; **Dompdf** para PDFs.
- **Contas e tokens** de Mercado Pago, Mercado Livre e Tiny por empresa; conta **SendGrid** para envio de e-mail.
- **Servidor web** com `mod_rewrite` (regras no `.htaccess`) e permissão de escrita na pasta `data/` (caches e logs).

12. Limitações e observações de segurança

- **Credenciais em texto no código:** tokens de Mercado Pago/Livre/Tiny, chave de API do SendGrid e credenciais de banco estão em claro no `config.php` e em alguns endpoints. O ideal é movê-los para variáveis de ambiente / arquivo fora do webroot, nunca versionados.
- **Validação de certificado desligada:** várias chamadas cURL usam `SSL_VERIFYPEER = false`. Em produção, o certificado deveria ser validado.
- **HTTPS não forçado:** a regra de redirecionamento para HTTPS está comentada no `.htaccess`.
- **Referências internas fixas:** há redirecionamento e referências a um IP de rede interna, próprias de ambiente de desenvolvimento.

- **Processamento sob demanda com teto de tempo:** períodos muito grandes podem exigir várias execuções do sync (o corte de ~110 s é proposital). Uma fila/cron evitaria isso.
- **Dependência das APIs externas:** limites de taxa e indisponibilidade do MP/ML/Tiny afetam a sincronização; há caches e *retries*, mas não um mecanismo de fila robusto.
- **Esquema evolutivo:** colunas novas entram por auto-migração em runtime — prático, mas o ideal é versionar migrações formalmente.

Documento gerado como briefing técnico do sistema MP Conciliador (Control Up), a partir da leitura do código-fonte. Valores de credenciais foram deliberadamente omitidos.